Project Overview

This project is a proof-of-concept (PoC) to **improve demand forecasts** using a combination of business knowledge and a Large Language Model (LLM). We start with a simple baseline forecast and then adjust it based on real-world factors like promotions, stockouts, distributor activity, and seasonal events. Finally, we measure how much accuracy improves.

---

1. Goals

- **Integrate Business Context:** Use structured data (from Excel or a mock database) about promotions, stockouts, distributor actions, and events to inform forecasts.
- **Enhance Forecasts:** Apply multipliers derived from context to a base forecast, with logic to avoid over-adjustment (diminishing returns).
- **Explain Changes:** Automatically generate clear, business-friendly explanations of why and how forecasts were adjusted.
- **Measure Uplift:** Compare the original (base) forecast to the adjusted (enhanced) forecast using accuracy metrics (MAPE, RMSE).

---

2. Data and Context Handling

2.1 ExcelFrappeDB (Real Data)

- **Loads an Excel file** containing rows of context data (each row has a type: promotion, stockout, distributor, or event).
- **Normalizes fields:** Converts text to lowercase, removes spaces, ensures date values follow a DD-MMM format.
- **Splits data** into separate lists: promotions, stockouts, distributor activities, and events.
- **Matches context** by product code, region, and ISO week number.

2.2 MockDatabase (Demo)

- A simple in-memory stub that returns **empty** or **hardcoded** context for testing without Excel.
- Allows quick setup for demonstrations or unit tests.

---

3. Core Tools and Agents

All major steps are wrapped in **tools** and **agents** (via CrewAI), or in a simplified interface for demos.

3.1 ContextFetcherTool

- Fetches context (promotions, stockouts, events, distributor activity) for a given product, region, and week.
- Formats it into an easy-to-read text summary.

3.2 ForecastCalculatorTool

- Takes a base forecast and a list of adjustment percentages (as decimals, e.g., 0.20 for +20%).
- **Multiplies** them cumulatively: adjusted = base * (1 + adj1) * (1 + adj2) ....
- **Rounds** the result and calculates percent change.

3.3 OllamaLLM

- A simple **wrapper** around the Ollama local LLM (llama3.2).
- Sends prompts and returns the model's text output or an error message.

3.4 Agents & Pipeline (CrewAI)

- **Context Agent:** Gathers raw context.
- **Enhancer Agent:** Uses context (and optionally LLM reasoning) to adjust the forecast.
- **Explanation Agent:** Generates a natural-language summary of adjustments and recommendations.
- Agents run **sequentially** in a Crew with three Task objects that pass outputs from one stage to the next.

---

4. MockInterface and Multipliers

For quick demos, the MockInterface class implements a manual_forecast_adjustment method:
1. **Extracts multipliers** and human-readable reasons from context:
- **Promotions:** multiplier = 1 + discount%/100.
- **Stockouts:** short (3–9 days) → +10%, long (10+ days) → +20%.
- **Distributor Activity:** maps levels (low, normal, high, very_high) to +5–20%.
- **Events:** maps impact (low, medium, high) to +5–20%.
2. **Diminishing Returns:** If there are more than two factors, reduce each extra factor by half of

its uplift.
3. **Applies all multipliers** to the base forecast in sequence.

---

5. Batch Processing and Reporting

- create_batch_forecast_processor() sets up a **batch processor** that:
  - Processes multiple requests at once.
  - Stores each result with product, region, week, base forecast, enhanced forecast, and reasons.
  - Assigns a confidence label (High if ≥2 reasons, Medium otherwise).
  - Exports results to a pandas DataFrame for further analysis.

---

6. Evaluation Metrics

ForecastEvaluator

- **Records** each scenario's forecasts and actual values.
- Calculates and returns:
  - **MAPE (Mean Absolute Percentage Error)** for both base and enhanced forecasts.
  - **RMSE (Root Mean Square Error)** for both.
- Also returns the full DataFrame with all per-scenario errors for deeper inspection.

---

7. User Interface (Gradio)

A simple web app to interact with the pipeline/demo:
- Inputs: Product code, region, week, base forecast, actual sales.
- On click:
1. Runs MockInterface.manual_forecast_adjustment.
2. Logs result in ForecastEvaluator.
3. Displays:
- **Forecast summary** (base, enhanced, reasons).
- **Evaluation metrics** (up-to-date MAPE and RMSE for both forecasts).
